

Detailed Content - H1 - H4

- [QMainWindow](#)
 - [Purpose & Common Use Cases](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Common Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QGridLayout](#)
 - [Purpose](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QFormLayout](#)
 - [Purpose](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QFileDialog](#)
 - [Purpose](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QMessageBox](#)
 - [Purpose](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QInputDialog](#)
 - [Purpose](#)

Detailed Content - H1 - H2

- [QMainWindow](#)
 - [Purpose & Common Use Cases](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Common Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QGridLayout](#)
 - [Purpose](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QFormLayout](#)
 - [Purpose](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QFileDialog](#)
 - [Purpose](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QMessageBox](#)
 - [Purpose](#)
 - [Key Features](#)
 - [Key Components](#)
 - [Common Methods](#)
 - [Signals and Slots](#)
 - [Integration with Other Widgets](#)
 - [Styling](#)
 - [Common Pitfalls](#)
 - [Best Practices](#)
- [QInputDialog](#)
 - [Purpose](#)

Detailed Content - H1

1. [QMainWindow](#)
2. [QGridLayout](#)
3. [QFormLayout](#)
4. [QFileDialog](#)
5. [QMessageBox](#)
6. [QInputDialog](#)
7. [QPixmap](#)
8. [QDockWidget](#)
9. [QScrollArea](#)
10. [QSpinBox](#)
11. [QDoubleSpinBox](#)
12. [QSplitter](#)
13. [QListView](#)
14. [QTableView](#)
15. [QTreeView](#)

- Key Features
- Key Components
- Common Methods
- Signals and Slots
 - Example Usage of Signals
- Integration with Other Widgets
- Styling
- Common Pitfalls
- Best Practices
- QPixmap
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QDockWidget
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QScrollArea
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QSpinBox
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QDoubleSpinBox
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QSplitter
 - Purpose
 - Key Features

- Key Features
- Key Components
- Common Methods
- Signals and Slots
- Integration with Other Widgets
- Styling
- Common Pitfalls
- Best Practices
- QPixmap
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QDockWidget
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QScrollArea
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QSpinBox
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QDoubleSpinBox
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- QSplitter
 - Purpose
 - Key Features
 - Key Components

- Key Components
- Common Methods
- Signals and Slots
- Integration with Other Widgets
- Styling
- Common Pitfalls
- Best Practices
- **QListView**
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- **QTableView**
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices
- **QTreeView**
 - Purpose
 - Key Features
 - Key Components
 - Common Methods
 - Signals and Slots
 - Integration with Other Widgets
 - Styling
 - Common Pitfalls
 - Best Practices

QMainWindow

Purpose & Common Use Cases

- **QMainWindow** is a **top-level window class** in Qt designed for building complex and full-featured application interfaces.
- It provides predefined areas like **menu bar**, **toolbars**, **status bar**, **dock widgets**, and **central widget**.
- Best suited for applications that require a well-structured and modular interface layout.

In VFX Tools, use it for:

- Building **tool launchers**, **editor shells**, or **asset browsers** with menus, dockable panels, and status messages.

- Creating **main control panels** for publishing or shot management tools.
- Structuring apps with **custom viewports, logs, and property editors** using dock widgets and central area.

Official Qt Documentation:

 [QMainWindow — Qt for Python](#)

Key Features

- **Menu Bar Support:** Built-in menu bar and actions.
- **Toolbars & Status Bar:** Easily add toolbars and bottom status messages.
- **Dock Widgets:** Add movable panels that can be torn off or docked.
- **Central Widget Area:** Main content view (e.g., timeline, render view, editor).
- **Keyboard Shortcuts & Actions:** Integrated action system with shortcuts and icons.

Key Components

- **Central Widget:** Main area (set via `setCentralWidget()`).
- **Menu Bar:** Top bar with menus (`menuBar()`).
- **Toolbars:** Horizontal or vertical quick access tools (`addToolBar()`).
- **Status Bar:** Bottom bar to show feedback/info (`statusBar()`).
- **Dock Widgets:** Detachable side panels (`addDockWidget()`).

Common Methods

- `setCentralWidget(widget)` → Sets the main content area.
- `menuBar()` → Accesses or creates the menu bar.
- `statusBar()` → Accesses or creates the status bar.
- `addToolBar(toolbar)` → Adds a toolbar.
- `addDockWidget(area, dockWidget)` → Adds a dockable panel.
- `setWindowTitle(title)` → Sets the title of the window.
- `resize(width, height)` → Resizes the window.

```

1 self.setCentralWidget(main_widget)           #
  setCentralWidget
2 menu = self.menuBar()                         # menuBar
3 self.statusBar().showMessage("Ready")         # statusBar
4 self.addToolBar(my_toolbar)                   # addToolBar
5 self.addDockWidget(Qt.LeftDockWidgetArea, my_dock_widget) #
  addDockWidget
6 self.setWindowTitle("VFX Tool")               #
  setWindowTitle
7 self.resize(800, 600)                         # resize
8

```

Common Signals and Slots

Handled through `QAction` objects:

- `QAction.triggered.connect()` – Connects menu or toolbar actions.
- `QMainWindow.closeEvent()` – Override to handle exit events.

```
1 action = QAction("Exit", self)
2 action.triggered.connect(self.close) # Triggered when clicked
```

Integration with Other Widgets

Frequently used with:

- `QDockWidget` (for side panels)
- `QToolBar` (for quick-access controls)
- `QLabel`, `QTextEdit`, `QTreeView`, or any custom widget as the central widget

```
1 from PySide2.QtWidgets import QTextEdit, QDockWidget
2
3 editor = QTextEdit()
4 dock = QDockWidget("Inspector")
5 dock.setWidget(QTextEdit("Properties"))
6 self.setCentralWidget(editor)
7 self.addDockWidget(Qt.RightDockWidgetArea, dock)
```

Styling

`QMainWindow` supports styling via its child widgets. Styling applies to **toolbars**, **menus**, and **status bar**, not the window frame itself.

```
1 self.setStyleSheet("""
2     QMainWindow {
3         background-color: #1e272e;
4     }
5     QMenuBar {
6         background-color: #2c3e50;
7         color: white;
8     }
9     QMenuBar::item:selected {
10         background-color: #00a8ff;
11     }
12 """)
```

Common Pitfalls

- **No Central Widget Set** – The window appears empty without it.
- **Missing Layout Inside Central Widget** – Central widget still needs a layout.
- **Overusing Dock Widgets** – Too many docks can clutter UI.
- **No Signal Hookups** – Menus or toolbars won't work without connected signals.

Best Practices

- **Use a Central Widget Layout** – Treat central area like a `QWidget`.
- **Organize with Docks** – Use docks for tools and side panels.
- **Group Actions with Toolbars & Menus** – Keep UI clean and consistent.
- **Use Status Bar for Feedback** – Display tips, loading states, etc.
- **Style Menus & Toolbars for Clarity** – Improve visual distinction

QGridLayout

Purpose

- QGridLayout is a layout manager that arranges child widgets in a **flexible grid** of rows and columns.
- It automatically handles spacing, alignment, and resizing behavior.
- Ideal for aligning widgets **horizontally and vertically** in structured patterns.

In VFX Tools, use it for:

- Creating organized UIs with **multiple input fields and labels**.
- Laying out **property panels**, render settings, or attribute editors.
- Grouping buttons and controls in a **clean matrix layout**.

Official Qt Documentation:

 [QGridLayout — Qt for Python](#)

Key Features

- Arranges widgets in a **grid format** using (row, column) positions.
- Supports **row/column spanning** to stretch widgets across multiple cells.
- Automatically manages layout resizing and widget alignment.
- Can be nested inside other layouts or parent widgets.

Key Components

- **Row & Column Indexes:** Used to place each widget precisely.
- **Row/Column Span:** Allows widgets to span multiple cells.
- **Stretch Factors:** Controls how space is distributed in resizing.
- **Alignment Flags:** Aligns widgets inside their grid cells.

Common Methods

- `addWidget(widget, row, column)` → Adds a widget to a specific grid cell.
- `addWidget(widget, row, column, rowSpan, colSpan)` → Adds widget across multiple cells.
- `setRowStretch(row, factor)` → Sets vertical space distribution.
- `setColumnStretch(column, factor)` → Sets horizontal space distribution.
- `setSpacing(pixels)` → Sets space between widgets.
- `setContentsMargins(left, top, right, bottom)` → Sets margins around the layout.

Example Usage of All Methods

```
1 layout = QGridLayout()
2 layout.addWidget(label1, 0, 0)                # addWidget
   (basic)
3 layout.addWidget(input_field, 0, 1, 1, 2)      # addWidget with
   span
4 layout.setRowStretch(1, 1)                     # setRowStretch
5 layout.setColumnStretch(2, 2)                  # setColumnStretch
6 layout.setSpacing(10)                          # setSpacing
7 layout.setContentsMargins(10, 10, 10, 10)      #
   setContentsMargins
8
```

Signals and Slots

QGridLayout itself **does not emit signals**, but its child widgets (e.g., buttons, sliders, input fields) do.

In a QGridLayout-based UI, you commonly connect signals from the **child widgets** like `QPushButton.clicked`, `QLineEdit.textChanged`, etc.

Integration with Other Widgets

QGridLayout is commonly used with:

- `QLabel`, `QLineEdit`, `QPushButton` — for form-style UIs.
- Can be used inside `QWidget`, `QGroupBox`, or nested inside another layout.

```
1 layout.addWidget(QLabel("Name:"), 0, 0)
2 layout.addWidget(QLineEdit(), 0, 1)
3
```

Styling

QGridLayout itself doesn't get styled, but you style **its child widgets**.

```
1 line_edit.setStyleSheet("""
2 QLineEdit {
3     background-color: #2e2e2e;
4     color: white;
5     border: 1px solid #666;
6 }
7 """)
8
```

Common Pitfalls

- Not setting row/column spans correctly → widgets may not appear as expected.
- Overlapping widgets in the same cell → one will overwrite the other.
- Stretch factors not used → layout might not resize properly.
- Styling the layout directly (only widgets inside can be styled).

Best Practices

- Use **row/column spans** for flexibility when needed.
- Combine with **stretch factors** to handle window resizing better.
- Nest QGridLayout inside `QVBoxLayout` or `QHBoxLayout` for complex UIs.
- Keep layouts readable by aligning labels and inputs in a consistent pattern.

QFormLayout

Purpose

- QFormLayout is used to create **label-field pairs** in a clean, vertical layout.
- It automatically aligns labels and fields without needing manual grid management.
- Ideal for building **forms, property panels, and input dialogs**.

In VFX Tools, use it for:

- Designing **submission forms** (e.g., shot name + status).
- Creating **render settings panels** with input fields.
- Structuring **property editors** for assets or effects.

Official Qt Documentation:

 [QFormLayout — Qt for Python](#)

Key Features

- Automatically aligns labels and corresponding fields.
- Supports adding widgets with or without labels.
- Provides control over row wrapping and spacing.
- Simplifies layout logic compared to QGridLayout for forms.

Key Components

- **Rows:** Each row typically has a label and a field widget.
- **Fields:** Can be any input widget (e.g., `QLineEdit`, `QComboBox`).
- **Form Items:** Represented as label-widget pairs internally.

Common Methods

- `addRow(label, field)` → Adds a row with label and widget.
- `insertRow(row, label, field)` → Inserts a row at a specific index.
- `addRow(widget)` → Adds a row with a single widget (no label).
- `removeRow(index or widget)` → Removes a specific row.
- `setSpacing(pixels)` → Sets vertical space between rows.
- `setContentsMargins(left, top, right, bottom)` → Sets layout margins.
- `labelForField(widget)` → Returns the label associated with a widget.

Example Usage of All Methods

```
1 layout = QFormLayout()
2 layout.addRow("Name:", QLineEdit())           # addRow(label,
3 layout.addRow(QPushButton("Browse"))           # addRow(widget
4 layout.insertRow(0, "Email:", QLineEdit())     # insertRow
5 layout.setSpacing(12)                          # setSpacing
6 layout.setContentsMargins(10, 10, 10, 10)      #
7 layout.removeRow(1)                            # removeRow by
8                                         index
```

Signals and Slots

QFormLayout itself **does not emit signals**, but its **child widgets** (e.g., `QLineEdit`, `QComboBox`, etc.) do.

You typically connect signals from child widgets like `textChanged`, `editingFinished`, etc.

Integration with Other Widgets

Frequently used with:

- `QLineEdit`, `QComboBox`, `QSpinBox` for inputs.
- `QPushButton` for actions (e.g., Submit, Cancel).
- Can be used inside `QGroupBox`, `QDialog`, or any `QWidget`.

```
1 layout.addRow("Render Quality:", QComboBox())
2 layout.addRow(QPushButton("Export"))
```

Styling

QFormLayout styling applies to **its child widgets**. You can style labels and fields individually.

```
1 line_edit.setStyleSheet("""
2 QLineEdit {
3     background-color: #2b2b2b;
4     color: white;
5     border: 1px solid #555;
6 }
7 """)
8
```

Common Pitfalls

- Forgetting to wrap input widgets correctly with `addRow()`.
- Trying to style the layout directly (only widgets can be styled).
- Not controlling margins/spacings in tight UIs.

Best Practices

- Use `QFormLayout` when you need **clean, label-aligned inputs**.
- Pair it with input widgets like `QLineEdit`, `QSpinBox`, etc.
- Keep consistent spacing and margins for better UI structure.
- Group related rows logically (use `QGroupBox` if needed).
- Keep labels short and meaningful to preserve space.

QFileDialog

Purpose

QFileDialog provides a native or custom dialog to **open files**, **save files**, or **select directories** through a user-friendly interface.

- Lets users browse the file system and choose files/folders.
- Supports filtering specific file types (like `.exr`, `.png`, `.mov`).
- Can be used both statically (quick use) and via dialog instances for more control.

In VFX Tools, use it for:

- File pickers in **render publishers** or **export tools**.
- Selecting input/output folders for **simulation caches**.
- Picking LUTs, plates, or EXRs for **shot review tools**.

 [Official Qt Documentation – QFileDialog](#)

Key Features

- File and directory selection (single/multiple).
- Filtering by file extension/type.
- Modal dialog behavior.
- Native system dialog or custom layout.
- Can emit signals when files/directories are selected.

Key Components

- Dialog Window: UI for file navigation.
- File Path: The selected path(s) from the user.
- Filters: Restrict visible file types/extensions.
- Mode: Whether it's for opening, saving, or choosing directories.

Common Methods

- `setFileMode(mode)` → Sets file selection mode (e.g. single, multiple).
- `setNameFilter(filter_string)` → Restricts visible files by extension.
- `selectFile(file_path)` → Preselects a file when the dialog opens.
- `setDirectory(directory_path)` → Sets the initial directory.
- `setViewMode(view_mode)` → Changes between list or detail view.
- `exec_()` → Executes the dialog modally.
- `setOption(option, enabled)` → Toggles various behaviors (e.g., show hidden files).

Example Usage of All Methods

```
1 from PySide2.QtWidgets import QFileDialog
2
3 dialog = QFileDialog()
4 dialog.setFileMode(QFileDialog.ExistingFiles)           # setFileMode
```

```

5 dialog.setNameFilter("Images (*.exr *.png *.jpg)")      # setNameFilter
6 dialog.selectFile("/path/to/default.exr")              # selectFile
7 dialog.setDirectory("/project/assets")                 # setDirectory
8 dialog.setViewMode(QFileDialog.Detail)                 # setViewMode
9 dialog.setOption(QFileDialog.ShowDirsOnly, True)       # setOption
10 dialog.exec_()                                         # exec_
11

```

Signals and Slots

- `fileSelected(str)` → Emitted when a single file is selected.
- `filesSelected(list)` → Emitted when multiple files are selected.
- `currentChanged(str)` → Emitted when the current file selection changes.
- `directoryEntered(str)` → Emitted when user navigates into a directory.
- `filterSelected(str)` → Emitted when the user changes the file filter.
- `urlSelected(QUrl)` → Emitted when a URL is selected.
- `urlsSelected(list)` → Emitted when multiple URLs are selected.

Example Usage of Signals

```

1 dialog = QFileDialog()
2 dialog.fileSelected.connect(lambda f: print("Selected:", f))      #
   fileSelected
3 dialog.filesSelected.connect(lambda fs: print("Multiple:", fs))   #
   filesSelected
4 dialog.currentChanged.connect(lambda f: print("Current file:", f)) #
   currentChanged
5 dialog.directoryEntered.connect(lambda d: print("Entered dir:", d)) #
   directoryEntered
6 dialog.filterSelected.connect(lambda fl: print("Filter:", fl))    #
   filterSelected
7

```

Integration with Other Widgets

`QFileDialog` works well with:

- **`QPushButton`** (to trigger the dialog)
- **`QLineEdit`** (to display the selected path)

```

1 self.button = QPushButton("Browse")
2 self.line_edit = QLineEdit()
3 self.button.clicked.connect(self.open_dialog)
4
5 def open_dialog(self):
6     file_path, _ = QFileDialog.getOpenFileName(self, "Select File")
7     if file_path:
8         self.line_edit.setText(file_path)

```

Styling

- `QFileDialog` can be **partially styled** when used as a custom dialog.
- Native dialogs (on Windows/macOS) do not fully support stylesheets.

```
1 dialog.setStyleSheet("QFileDialog { font-size: 14px; })
```

Common Pitfalls

- Forgetting to check if the returned file path is empty before using it.
- Not setting the parent widget → may cause dialog to float or behave unpredictably.
- Expecting full styling support on native dialogs — it's limited.
- Using static methods and expecting signals (they don't emit signals).

Best Practices

- Always pass `self` as parent to maintain modal and cleanup behavior.
- Use filters to improve usability and prevent user errors.
- Always check if a file path was selected before proceeding.
- Use dialog instances when you need access to signals or want customization.
- Keep UI feedback clear when a file is selected.

QMessageBox

Purpose

- QMessageBox is used to display information, warnings, errors, or confirmation dialogs to the user.
- It helps guide user decisions by presenting clear messages with standard buttons.
- It provides immediate feedback for important events in an application.

In VFX Tools, use it for:

- Confirming **render submissions**.
- Showing **errors in asset publishing**.
- Warning users before **deleting shots, caches, or versions**.

Official Qt Documentation:

 [QMessageBox — Qt for Python](#)

Key Features

- Ready-made dialog types: Information, Warning, Critical, Question.
- Standard buttons: OK, Cancel, Yes, No, Retry, etc.
- Modal behavior that blocks the parent window.
- Built-in icons for visual clarity.
- Can be used as static calls or full dialog instances.

Key Components

- Message Text: Main message shown to the user.
- Informative Text: Optional secondary explanation.
- Icon: Visual indicator of message type.
- Buttons: User response choices.

- Return Value: Indicates which button the user clicked.

Common Methods

- `setText(text)` → Sets the main message.
- `setInformativeText(text)` → Sets secondary descriptive text.
- `setIcon(icon_type)` → Sets the message icon.
- `setStandardButtons(buttons)` → Adds standard buttons.
- `setDefaultButton(button)` → Sets the default selected button.
- `exec_()` → Displays the dialog modally and waits for user response.

Example Usage of All Methods

```
1 msg = QMessageBox()
2 msg.setText("Render completed")           # setText
3 msg.setInformativeText("All frames rendered.") # setInformativeText
4 msg.setIcon(QMessageBox.Information)      # setIcon
5 msg.setStandardButtons(QMessageBox.Ok | QMessageBox.Cancel) #
   setStandardButtons
6 msg.setDefaultButton(QMessageBox.Ok)      # setDefaultButton
7 msg.exec_()                              # exec_
8
```

Signals and Slots

`QMessageBox` does not emit many direct signals, but common interaction is handled via **return values**.

- `buttonClicked(QAbstractButton*)` → Emitted when any button is clicked.

Example Usage of Signals

```
1 msg = QMessageBox()
2 msg.buttonClicked.connect(lambda b: print("Button clicked:", b.text()))
   # buttonClicked
3 msg.exec_()
4
```

Integration with Other Widgets

`QMessageBox` is usually triggered by:

- `QPushButton` (confirmation actions)
- `QTableView` (delete or overwrite warnings)

```
1 self.delete_btn.clicked.connect(self.confirm_delete)
2
3 def confirm_delete(self):
4     QMessageBox.information(self, "Info", "Delete completed")
5
```

Styling

`QMessageBox` supports limited styling.

```
1 msg.setStyleSheet("QMessageBox { font-size: 14px; })")
```

Common Pitfalls

- Forgetting to capture the return value from `exec_()`.
- Using too many buttons and confusing the user.
- Not setting a default button.
- Displaying long technical messages to artists.

Best Practices

- Use clear, concise messages.
- Choose icons consistently for error/warning/info.
- Always set a default button.
- Avoid interrupting users unnecessarily.
- Keep dialogs short and action-focused.

QInputDialog

Purpose

- QInputDialog provides a quick way to **get simple input** from the user (like text, numbers, or selections).
- It's used for collecting lightweight data **without creating a full form**.
- Works well for single-question prompts or quick configuration entries.

In VFX Tools, use it for:

- Letting users **enter version names** or **shot comments**.
- Choosing a **frame range** or **render resolution** on-the-fly.
- Prompting for **user initials** in shot tracking tools.

Official Qt Documentation:

 [QInputDialog — Qt for Python](#)

Key Features

- Built-in input types: text, integers, floats, combo box selections.
- Modal behavior: blocks parent window until response is given.
- Supports static method calls for quick use.
- Returns the value entered and a status flag (OK/Cancel).

Key Components

- Prompt Label: The question shown to the user.
- Input Field: Accepts text, numbers, or options.
- OK/Cancel Buttons: Allow confirming or canceling the input.
- Return Value: What the user typed or selected.

Common Methods

- `getText(parent, title, label)` → Prompts user to enter text.
- `getInt(parent, title, label)` → Prompts user for an integer.

- `getDouble(parent, title, label)` → Prompts user for a float.
- `getItem(parent, title, label, items)` → Prompts user to select from a list.

Example Usage of All Methods

```

1 # getText
2 text, ok = QInputDialog.getText(self, "Input", "Enter your name:")
3
4 # getInt
5 value, ok = QInputDialog.getInt(self, "Input", "Enter frame number:")
6
7 # getDouble
8 number, ok = QInputDialog.getDouble(self, "Input", "Enter opacity
   value:")
9
10 # getItem
11 item, ok = QInputDialog.getItem(self, "Select Engine", "Choose Render
   Engine:", ["Arnold", "Redshift", "VRay"])
12

```

Signals and Slots

`QInputDialog` can be used as a standalone dialog instance, which supports:

- `textValueChanged(str)` → Emitted when the text input is changed.
- `textValueSelected(str)` → Emitted when the text is selected and accepted.
- `intValueChanged(int)` → Emitted when the integer value changes.
- `doubleValueChanged(float)` → Emitted when the double value changes.

Example Usage of Signals

```

1 dialog = QInputDialog(self)
2 dialog.setLabelText("Enter shot number:")
3 dialog.textValueChanged.connect(lambda text: print("Changed:", text))
   # textValueChanged
4 dialog.textValueSelected.connect(lambda text: print("Selected:", text))
   # textValueSelected
5 dialog.exec_()
6

```

Integration with Other Widgets

`QInputDialog` is often triggered by:

- `QPushButton` for initiating input.
- `QLineEdit` to store the result.

```

1 self.button = QPushButton("Enter Version")
2 self.line_edit = QLineEdit()
3 self.button.clicked.connect(self.ask_for_version)
4
5 def ask_for_version(self):
6     version, ok = QInputDialog.getText(self, "Input", "Enter version:")
7     if ok:
8         self.line_edit.setText(version)
9

```

Styling

- Limited styling is supported when used as an instance.
- Styling has no effect on static method calls.

```
1 dialog.setStyleSheet("QInputDialog { font-size: 14px; }")
2
```

Common Pitfalls

- Forgetting to check the `ok` return flag — user may cancel the dialog.
- Using static methods when signals or styling are needed.
- Using it for complex forms — better handled by custom dialogs.
- Not providing clear and simple labels for the prompt.

Best Practices

- Always check the `ok` flag before using the return value.
- Use `getItem()` for predefined options to reduce typing errors.
- Provide helpful and concise labels to guide user input.
- Use static methods for quick input; use dialog instance when more control is needed.
- Avoid overusing input dialogs for every minor input — use where appropriate.

QPixmap

Purpose

- QPixmap is used to **handle and display images** in PySide2 applications.
- It is optimized for **on-screen rendering**, making it ideal for GUI work.
- Commonly used to load, scale, and display image data in widgets.

In VFX Tools, use it for:

- Displaying **image previews** (e.g., thumbnails, render outputs).
- Showing **plate references or LUT previews** in tools.
- Caching and displaying images in **review or shot tracking panels**.

Official Qt Documentation:

 [QPixmap — Qt for Python](#)

Key Features

- Load images from files or in-memory buffers.
- Supports common formats: `.png`, `.jpg`, `.bmp`, `.tiff`, etc.
- Optimized for GUI display and fast rendering.
- Can be used directly in QLabel, QPainter, or custom widgets.
- Easily scaled or transformed without modifying the original.

Key Components

- Source Path: File path or resource to load the image from.
- QPixmap Object: The actual image data container.

- Display Widget: QLabel or other widget used to show the image.

Common Methods

- `load(filename)` → Loads an image from file path.
- `isNull()` → Checks if the pixmap contains valid image data.
- `scaled(width, height, mode)` → Returns a scaled version of the pixmap.
- `width() / height()` → Returns the dimensions of the pixmap.
- `save(filename)` → Saves the pixmap to a file.
- `copy(x, y, w, h)` → Returns a cropped copy of the pixmap.

Example Usage of All Methods

```
1 pixmap = QPixmap()  
2 pixmap.load("render.jpg")           # load  
3 if not pixmap.isNull():              # isNull  
4     scaled = pixmap.scaled(200, 200) # scaled  
5     print(pixmap.width(), pixmap.height()) # width, height  
6     cropped = pixmap.copy(50, 50, 100, 100) # copy  
7     cropped.save("cropped.jpg")         # save  
8
```

Signals and Slots

QPixmap itself has **no native signals**, but we commonly use signals from connected widgets like:

Commonly Used Signals

- `QPushButton.clicked` → To trigger image loading.
- `QLabel.linkActivated(str)` → Used when QLabel contains clickable HTML (optional).
- `QFileDialog.fileSelected(str)` → When selecting an image path from a dialog.

Example Usage of Signals

```
1 self.button.clicked.connect(self.load_image) # Trigger from  
  QPushButton  
2 dialog = QFileDialog(self)  
3 dialog.fileSelected.connect(lambda path: print("File selected:", path))  
  # QFileDialog signal  
4
```

Integration with Other Widgets

QPixmap is commonly integrated with:

- `QLabel` to display the image.
- `QPushButton` to load or refresh image content.

```
1 pixmap = QPixmap("plate.jpg")  
2 label = QLabel()  
3 label.setPixmap(pixmap) # Display image inside QLabel  
4
```

Styling

QPixmap itself is not styled, but **its container widget** (e.g. QLabel) can be styled.

```
1 label.setStyleSheet("QLabel { background-color: black; border: 1px  
solid #ccc; }")
```

Common Pitfalls

- Using QPixmap in non-GUI threads (QPixmap is **not thread-safe**).
- Not checking `isNull()` after loading → leads to crashes or blank labels.
- Trying to style QPixmap directly instead of the display widget.
- Loading unsupported image formats.

Best Practices

- Always check if the pixmap is valid using `isNull()`.
- Use `scaled()` to display thumbnails instead of resizing QLabel directly.
- Store and reuse loaded QPixmap's to avoid performance hits.
- Perform all QPixmap operations in the GUI thread only.
- Style QLabel or QWidget containers — not QPixmap itself.

QDockWidget

Purpose

- QDockWidget is used to create **dockable panels** that can be attached, detached, or rearranged around a main window.
- It allows users to **customize their workspace layout** dynamically.
- It helps organize tools and information into logical UI sections.

In VFX Tools, use it for:

- Building **node editors, asset browsers, and property panels**.
- Creating **timeline, render settings, and shot info panels**.
- Allowing artists to rearrange tool layouts based on personal workflow.

Official Qt Documentation:

 [QDockWidget — Qt for Python](#)

Key Features

- Dockable and floating panel behavior.
- Supports multiple docking areas.
- Can be tabbed together with other dock widgets.
- Can be locked, hidden, or floated by the user.
- Integrates tightly with QMainWindow.

Key Components

- Dock Widget: The container panel.
- Title Bar: Shows dock name and controls.
- Dock Areas: Left, Right, Top, Bottom regions of the main window.
- Contained Widget: The actual tool or UI placed inside the dock.

Common Methods

- `setWidget(widget)` → Assigns the main widget inside the dock.
- `setAllowedAreas(areas)` → Controls where the dock can be placed.
- `setFloating(bool)` → Toggles floating state.
- `setVisible(bool)` → Shows or hides the dock.
- `setFeatures(features)` → Enables/disables close, float, move controls.
- `setWindowTitle(title)` → Sets dock title.
- `toggleViewAction()` → Returns an action for show/hide toggling.

Example Usage of All Methods

```
1 dock = QDockWidget("Tool Panel")
2 dock.setWidget(QWidget()) # setWidget
3 dock.setAllowedAreas(Qt.LeftDockWidgetArea | Qt.RightDockWidgetArea) #
  setAllowedAreas
4 dock.setFloating(False) #
  setFloating
5 dock.setVisible(True) # setVisible
6 dock.setFeatures(QDockWidget.DockWidgetMovable |
  QDockWidget.DockWidgetClosable) # setFeatures
7 dock.setWindowTitle("Inspector") #
  setWindowTitle
8 action = dock.toggleViewAction() #
  toggleViewAction
9
```

Signals and Slots

QDockWidget provides useful signals for layout management:

- `visibilityChanged(bool)` → Emitted when dock is shown or hidden.
- `topLevelChanged(bool)` → Emitted when dock becomes floating or docked.
- `dockLocationChanged(Qt.DockWidgetArea)` → Emitted when moved to a new area.

Example Usage of Signals

```
1 dock.visibilityChanged.connect(lambda v: print("Visible:", v))
  # visibilityChanged
2 dock.topLevelChanged.connect(lambda f: print("Floating:", f))
  # topLevelChanged
3 dock.dockLocationChanged.connect(lambda a: print("Moved to area:", a))
  # dockLocationChanged
4
```

Integration with Other Widgets

QDockWidget is always used with `QMainWindow` and commonly contains:

- `QTreeView`, `QTableView`, `QListView`
- `QWidget` with custom layouts

```
1 main_window.addDockWidget(Qt.RightDockWidgetArea, dock)
```

Styling

`QDockWidget` and its title bar support visual styling.

```
1 dock.setStyleSheet("""
2 QDockWidget {
3     background-color: #2b2b2b;
4     color: #ffffff;
5 }
6 QDockWidget::title {
7     background-color: #444444;
8     padding: 6px;
9 }
10 """)
```

Common Pitfalls

- Not using `QMainWindow` as the main container.
- Forgetting to set a widget inside the dock.
- Allowing docks to float when the tool requires a fixed layout.
- Overcrowding the UI with too many docks.

Best Practices

- Group related tools into logical dock panels.
- Save and restore dock layout for user sessions.
- Provide toggle actions for showing/hiding docks.
- Use consistent naming for dock titles.
- Avoid clutter by limiting visible docks at startup.

QScrollArea

Purpose

- `QScrollArea` provides a scrollable view for widgets that **exceed the visible space**.
- It enables horizontal and/or vertical scrolling of large content areas.
- It is used when layout space is limited but content is dynamic or complex.

In VFX Tools, use it for:

- Scrolling through long **node or graph editors**.
- Displaying **asset lists or property panels** in compact UIs.
- Hosting **timeline sliders or history panels** inside constrained spaces.

Official Qt Documentation:

 [QScrollArea — Qt for Python](#)

Key Features

- Automatically provides scrollbars when content exceeds size.
- Supports scrollable widgets of any type.
- Allows control over scroll directions and scrollbar policies.
- Can resize child widget dynamically or lock to fixed size.
- Useful for embedding nested UIs in scrollable regions.

Key Components

- Scroll Area: The container widget.
- Widget to Scroll: The actual content inside the scroll area.
- Scrollbars: Automatically shown or hidden based on content size and policy.

Common Methods

- `setWidget(widget)` → Sets the content widget to be scrolled.
- `setWidgetResizable(bool)` → Enables resizing of the content widget.
- `widget()` → Returns the widget being displayed inside the scroll area.
- `ensureVisible(x, y)` → Scrolls to ensure a point is visible.
- `setHorizontalScrollBarPolicy(policy)` → Sets horizontal scrollbar behavior.
- `setVerticalScrollBarPolicy(policy)` → Sets vertical scrollbar behavior.

Example Usage of All Methods

```
1 scroll = QScrollArea()
2 scroll.setWidget(content_widget)           # setWidget
3 scroll.setWidgetResizable(True)           # setWidgetResizable
4 child = scroll.widget()                    # widget()
5 scroll.ensureVisible(100, 200)             # ensureVisible
6 scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAsNeeded) #
  setHorizontalScrollBarPolicy
7 scroll.setVerticalScrollBarPolicy(Qt.ScrollBarAsNeeded)   #
  setVerticalScrollBarPolicy
8
```

Signals and Slots

- QScrollArea does not define its own signals directly, but you can use:
- Commonly Used Signals with QScrollArea:
 - `QScrollBar.valueChanged(int)` → From the scroll area's scrollbars.
 - `QPushButton.clicked()` → Often used to update or scroll content.
 - `QWidget.resizeEvent()` → Used for custom scroll behavior.

Example Usage of Signals

```
1 scroll.verticalScrollBar().valueChanged.connect(lambda v:
  print("Scrolled to:", v)) # ScrollBar signal
2 button.clicked.connect(lambda: scroll.ensureVisible(0, 500))
  # Scroll to position
```

Integration with Other Widgets

QScrollArea can host:

- Custom QWidget layouts (forms, panels, image viewers)
- QLabel, QFormLayout, QVBoxLayout inside a scrollable QWidget

```
1 scroll_area.setWidgetResizable(True)
2 scroll_area.setWidget(your_custom_widget)
3
```

Styling

QScrollArea and its scrollbars support full styling via CSS.

```
1 scroll_area.setStyleSheet("""
2 QScrollArea {
3     background-color: #2c2c2c;
4 }
5 QScrollBar:vertical {
6     background: #444;
7     width: 10px;
8 }
9 """)
10
```

Common Pitfalls

- Forgetting to set `setWidgetResizable(True)` → content won't adapt to resizing.
- Adding layouts directly to QScrollArea instead of placing them inside a widget.
- Over-nesting scroll areas leads to performance drops.
- Styling not applying if scrollbars are not visible at runtime.

Best Practices

- Wrap layouts inside a QWidget, then place that into the scroll area.
- Enable `setWidgetResizable(True)` for responsive designs.
- Use `ensureVisible()` to programmatically scroll to dynamic areas.
- Avoid scroll clutter by managing scroll direction based on use case.
- Style scrollbars to match your UI theme for better visual consistency.

QSpinBox

Purpose

- QSpinBox allows users to **input and adjust integer values** using buttons or direct typing.
- It ensures that input stays within a **defined numeric range**.
- It simplifies numeric entry and prevents invalid values.

In VFX Tools, use it for:

- Setting **frame numbers** and **frame ranges**.
- Adjusting **sample counts**, **iterations**, or **passes**.
- Controlling **render resolution parameters** and quality levels.

Official Qt Documentation:

 [QSpinBox — Qt for Python](#)

Key Features

- Restricts input to integers only.
- Provides step-based increase/decrease controls.
- Supports custom ranges and default values.
- Can display prefixes, suffixes, and special value text.
- Emits signals whenever the value changes.

Key Components

- Display Field: Shows the current numeric value.
- Up/Down Buttons: Increase or decrease the value.
- Range Limits: Minimum and maximum allowed values.

Common Methods

- `setValue(value)` → Sets the current value.
- `value()` → Returns the current value.
- `setRange(min, max)` → Sets allowed value range.
- `setSingleStep(step)` → Sets step increment size.
- `setPrefix(text)` → Adds text before the number.
- `setSuffix(text)` → Adds text after the number.
- `setSpecialValueText(text)` → Text shown for minimum value.

Example Usage of All Methods

```
1 spin = QSpinBox()
2 spin.setRange(0, 100)           # setRange
3 spin.setValue(10)              # setValue
4 spin.setSingleStep(5)          # setSingleStep
5 spin.setPrefix("Frame ")       # setPrefix
6 spin.setSuffix(" px")          # setSuffix
7 spin.setSpecialValueText("Auto") # setSpecialValueText
8 current = spin.value()         # value
9
```

Signals and Slots

Commonly Used Signals

- `valueChanged(int)` → Emitted when value changes.
- `editingFinished()` → Emitted when user finishes editing.

Example Usage of Signals

```
1 spin.valueChanged.connect(lambda v: print("Value:", v)) # valueChanged
2 spin.editingFinished.connect(lambda: print("Editing done")) #
  editingFinished
3
```

Integration with Other Widgets

Often integrated with:

- **QSlider** for synchronized numeric control.
- **QLabel** for displaying descriptive text.

```
1 slider.valueChanged.connect(spin.setValue)
2 spin.valueChanged.connect(slider.setValue)
3
```

Styling

QSpinBox supports styling for font, colors, borders, and buttons.

```
1 spin.setStyleSheet("""
2 QSpinBox {
3     font-size: 14px;
4     color: #ffffff;
5     background-color: #2b2b2b;
6     border: 1px solid #666;
7 }
8 """)
9
```

Common Pitfalls

- Forgetting to set a valid range.
- Not synchronizing dependent controls.
- Using QSpinBox when floating-point values are required (use QDoubleSpinBox instead).
- Overusing prefix/suffix causing cluttered UI.

Best Practices

- Define sensible default ranges.
- Always validate value changes before using them in calculations.
- Keep step size intuitive for artists.
- Synchronize related widgets.
- Style consistently with the rest of the UI.

QDoubleSpinBox

Purpose

- QDoubleSpinBox allows users to **input and adjust floating-point values** using increment buttons or typing.
- It restricts input to a **valid numerical range with decimal precision**.
- Ideal when you need **precise control over non-integer values** in UI.

In VFX Tools, use it for:

- Controlling **opacity, exposure, gamma, or gain** values.
- Adjusting **scale factors, resolution multipliers**, or **percentages**.
- Providing **fine-tuned numeric sliders** in lighting or comp tools.

Official Qt Documentation:

 [QDoubleSpinBox — Qt for Python](#)

Key Features

- Accepts floating-point values with custom step size and precision.
- Displays decimal places and limits input to defined min/max range.
- Offers optional prefixes, suffixes, and special labels.
- Supports keyboard input and up/down arrows for adjustment.
- Emits signals when values change or editing is complete.

Key Components

- Value Display: Editable numeric field with decimal precision.
- Up/Down Buttons: For increasing or decreasing value.
- Range and Step: Minimum, maximum, and step size.
- Formatting: Prefix, suffix, and decimals shown.

Common Methods

- `setValue(float)` → Sets the current value.
- `value()` → Returns the current value.
- `setRange(min, max)` → Sets minimum and maximum limits.
- `setSingleStep(step)` → Sets the increment value per step.
- `setDecimals(count)` → Sets number of decimal places shown.
- `setPrefix(text)` → Adds text before the value.
- `setSuffix(text)` → Adds text after the value.
- `setSpecialValueText(text)` → Shows text for special values.

Example Usage of All Methods

```
1 spin = QDoubleSpinBox()
2 spin.setRange(0.0, 10.0)           # setRange
3 spin.setValue(1.25)                # setValue
4 spin.setSingleStep(0.1)            # setSingleStep
5 spin.setDecimals(2)                # setDecimals
6 spin.setPrefix("Opacity: ")        # setPrefix
7 spin.setSuffix(" %")               # setSuffix
8 spin.setSpecialValueText("Auto")    # setSpecialValueText
9 current = spin.value()              # value
10
```

Signals and Slots

Commonly Used Signals

- `valueChanged(float)` → Emitted when the value changes.
- `editingFinished()` → Emitted when editing is completed.

Example Usage of Signals

```

1 def on_value_changed(val):
2     print("New value:", val)
3
4 def on_edit_finished():
5     print("Editing finished")
6
7 spin.valueChanged.connect(on_value_changed)      # valueChanged
8 spin.editingFinished.connect(on_edit_finished)   # editingFinished
9

```

Integration with Other Widgets

`QDoubleSpinBox` is often paired with:

- `QSlider` (for dragging-based adjustment)
- `QLabel` (for context or descriptions)

```

1 def slider_changed(value):
2     spin.setValue(value / 100.0)
3
4 def spinbox_changed(value):
5     slider.setValue(int(value * 100))
6
7 slider.valueChanged.connect(slider_changed)
8 spin.valueChanged.connect(spinbox_changed)
9

```

Styling

`QDoubleSpinBox` supports full widget styling (font, color, border, buttons, etc.).

```

1 spin.setStyleSheet("""
2 QDoubleSpinBox {
3     font-size: 14px;
4     color: #ffffff;
5     background-color: #2e2e2e;
6     border: 1px solid #555;
7 }
8 """)
9

```

Common Pitfalls

- Using integers when floating-point control is needed — use `QDoubleSpinBox` for decimal inputs.
- Forgetting to set decimal precision → numbers appear rounded.
- Not syncing values correctly with sliders or other controls.
- Using too high a step value for precise control.

Best Practices

- Always define `setDecimals()` clearly to match your precision needs.
- Sync with sliders to allow both drag and precise input.

- Use suffixes like `%`, `px`, or `units` for clarity.
- Validate user input or clamp it in production tools.
- Style the widget to match your UI scheme.

QSplitter

Purpose

- QSplitter is used to **divide space between multiple widgets** with a draggable splitter handle.
- It allows users to **resize sections of a UI** dynamically.
- Useful for creating **adjustable panels** and layout customization.

In VFX Tools, use it for:

- Creating **layout-resizable node editors, file explorers, or viewer panels**.
- Splitting a UI into **preview + properties** or **timeline + viewport** sections.
- Providing a **drag-to-resize control** in tool panels.

Official Qt Documentation:

 [QSplitter — Qt for Python](#)

Key Features

- Arranges widgets horizontally or vertically.
- Users can resize widgets by dragging the splitter handle.
- Supports nested splitters for complex layouts.
- Can set initial and minimum sizes of widgets.
- Splitter position can be saved and restored.

Key Components

- Orientation: `Qt.Horizontal` or `Qt.Vertical`.
- Child Widgets: Widgets added to the splitter.
- Split Handle: UI element between widgets for resizing.
- Sizes: Adjustable using pixel values.

Common Methods

- `addWidget(widget)` → Adds a widget to the splitter.
- `setOrientation(Qt.Orientation)` → Sets horizontal or vertical layout.
- `setSizes([size1, size2, ...])` → Sets initial sizes of child widgets.
- `sizes()` → Returns current sizes of the splitter widgets.
- `setStretchFactor(index, factor)` → Controls resizing behavior.
- `widget(index)` → Returns the widget at the given index.
- `count()` → Returns number of widgets in splitter.

Example Usage of All Methods

```

1 splitter = QSplitter()
2 splitter.setOrientation(Qt.Horizontal)      # setOrientation
3 splitter.addWidget(widget1)                 # addWidget
4 splitter.addWidget(widget2)
5 splitter.setSizes([200, 300])              # setSizes
6 current_sizes = splitter.sizes()           # sizes
7 splitter.setStretchFactor(0, 1)            # setStretchFactor
8 widget_at_index = splitter.widget(0)       # widget
9 widget_count = splitter.count()            # count
10

```

Signals and Slots

QSplitter uses signals from its **child widgets**. Most common ones used with QSplitter-based UIs:

Commonly Used Signals

- `QLineEdit.textChanged(str)` → Child widget input updates.
- `QListView.clicked(QModelIndex)` → Triggered in explorer/view panels.
- `QSplitter.splitterMoved(int, int)` → Emitted when handle is moved.

Example Usage of Signals

```

1 def on_splitter_moved(pos, index):
2     print("Splitter moved. Position:", pos, "Index:", index)
3
4 splitter.splitterMoved.connect(on_splitter_moved) # splitterMoved
5

```

Integration with Other Widgets

Commonly paired with:

- `QTextEdit`, `QTreeView`, `QListView`, `QTableView`, `QLabel`
- Other nested `QSplitter` instances

```

1 splitter.addWidget(QListView())
2 splitter.addWidget(QTextEdit())
3

```

Styling

QSplitter supports styling of its handle via CSS.

```

1 splitter.setStyleSheet("""
2 QSplitter::handle {
3     background-color: #444;
4     width: 6px;
5 }
6 """)
7

```

Common Pitfalls

- Not setting stretch factors → some widgets won't resize properly.
- Using fixed sizes for widgets inside splitter reduces flexibility.
- Forgetting to store sizes if restoring UI state is needed.
- Over-nesting splitters → results in cluttered and complex UI.

Best Practices

- Use `setStretchFactor()` to define resizing priority.
- Initialize with meaningful default sizes via `setSizes()`.
- Combine `QSplitter` with layout-based widgets for flexibility.
- Avoid placing too many widgets in a single splitter.
- Style splitter handles subtly to match your UI.

QListView

Purpose

- QListView is used to **display and manage a vertical list of items** based on a model/view architecture.
- It separates **data (model)** from its **visual representation (view)**.
- Ideal when data needs to be dynamically updated, filtered, or styled.

In VFX Tools, use it for:

- Browsing through **shot names, render versions, or assets**.
- Building **selection lists**, such as render layers or AOVs.
- Integrating with models for **dynamic content updates** (e.g., live publish tracking).

Official Qt Documentation:

 [QListView — Qt for Python](#)

Key Features

- **Model/View Architecture:** Separates data from UI — great for scalability.
- **Custom Models:** Supports `QStringListModel`, custom models, or `QStandardItemModel`.
- **Custom Delegates:** Render items with custom styles or widgets.
- **Single/Multiple Selection:** Control item interaction mode.
- **Efficient Performance:** Optimized for large, dynamic lists.

Key Components

- **View:** Displays the data visually.
- **Model:** Supplies data (e.g., strings, items, or a database).
- **Delegate (optional):** Custom renderer/editor for each item.
- **Selection Model:** Manages selection behavior.

Common Methods

- `setModel(model)` → Sets the model containing the data.
- `model()` → Returns the model currently set.
- `setEditTriggers(triggers)` → Controls how items can be edited.
- `setSelectionMode(mode)` → Defines how items can be selected.
- `setViewMode(mode)` → Changes display mode (List or Icon view).

- `selectedIndexes()` → Returns a list of selected items.

Example Usage of All Methods

```

1 view = QListView()
2 model = QStringListModel(["Render", "Compositing", "Lighting"])
3 view.setModel(model) # setModel
4 current_model = view.model() # model
5 view.setEditTriggers(QListView.DoubleClicked) # setEditTriggers
6 view.setSelectionMode(QListView.SingleSelection) # setSelectionMode
7 view.setViewMode(QListView.ListMode) # setViewMode
8 selected = view.selectedIndexes() # selectedIndexes
9

```

Signals and Slots

- `clicked(index)` → Emitted when an item is clicked.
- `doubleClicked(index)` → Emitted when an item is double-clicked.
- `activated(index)` → Triggered by Enter/Return or double-click.
- `pressed(index)` → Emitted when mouse is pressed on an item.

Example Usage of Signals

```

1 def on_item_clicked(index):
2     print("Item clicked:", index.data())
3
4 view.clicked.connect(on_item_clicked) # clicked
5

```

Integration with Other Widgets

Often paired with:

- `QStringListModel`, `StandardItemModel` for data.
- `QPushButton` for add/remove operations.
- `QLabel`, `QTextEdit` for displaying selected content.

```

1 button.clicked.connect(lambda: model.setStringList(["Updated",
2 "List"]))

```

Styling

`QListView` supports full styling for items and selection.

```

1 view.setStyleSheet("""
2 QListView {
3     background-color: #2b2b2b;
4     color: #ffffff;
5     font-size: 14px;
6     border: 1px solid #555;
7 }
8 QListView::item:selected {
9     background-color: #007acc;
10 }
11 """)
12

```

Common Pitfalls

- Forgetting to assign a model → nothing shows in the list.
- Not handling the index properly in signal functions (`index.data()` is required).
- Attempting to manually add items without using the model.
- Editing triggers not enabled → item edits don't activate.

Best Practices

- Use `QStringListModel` for simple lists, `QStandardItemModel` for more control.
- Always connect index signals using `index.data()` to access the text.
- Make UI responsive by styling selections and hover states.
- Keep model and view logic separate for clarity.
- Use `selectedIndexes()` for item processing or batch operations.

QTableView

Purpose

- QTableView provides a **tabular interface** for displaying and interacting with data models.
- It is part of Qt's **Model/View framework**, where data is managed separately from how it's displayed.
- Suitable for complex, multi-column structured data in interactive tools.

In VFX Tools, use it for:

- Displaying **shot lists**, **version histories**, or **render queues**.
- Building **editable property tables** for assets or tasks.
- Showing data from **databases or publishing systems** in real-time.

Official Qt Documentation:

 [QTableView — Qt for Python](#)

Key Features

- Supports **multi-column tabular data** using models.
- Offers features like sorting, selection, editing, and custom formatting.
- Integrates with `QAbstractItemModel`, `QStandardItemModel`, and others.
- Highly customizable using delegates and headers.

Key Components

- **Model:** Supplies the actual data.
- **View:** QTableView handles the display and interaction.
- **Delegate (optional):** Customizes how items are drawn or edited.
- **Headers:** Optional row and column labels.

Common Methods

- `setModel(model)` → Assigns the data model.
- `model()` → Returns the assigned model.
- `setEditTriggers(triggers)` → Defines how cells are edited.
- `setSelectionBehavior(behavior)` → Determines if selection is by row or cell.
- `setSelectionMode(mode)` → Controls single/multiple selection.
- `setSortingEnabled(bool)` → Enables/disables sorting.
- `selectRow(index)` → Selects a specific row.
- `horizontalHeader()` / `verticalHeader()` → Access table headers.
- `setColumnWidth(col, width)` → Sets the width of a column.
- `resizeColumnsToContents()` → Auto-fits all columns.

Example Usage of All Methods

```
1 table = QTableView()
2 model = QStandardItemModel(3, 2)
3 table.setModel(model) # setModel
4 table.setEditTriggers(QTableView.DoubleClicked) # setEditTriggers
5 table.setSelectionBehavior(QTableView.SelectRows) #
  setSelectionBehavior
6 table.setSelectionMode(QTableView.SingleSelection) # setSelectionMode
7 table.setSortingEnabled(True) # setSortingEnabled
8 table.selectRow(1) # selectRow
9 table.setColumnWidth(0, 150) # setColumnWidth
10 table.resizeColumnsToContents() #
   resizeColumnsToContents
11
```

Signals and Slots

- `clicked(index)` → Triggered when a cell is clicked.
- `doubleClicked(index)` → Triggered when a cell is double-clicked.
- `activated(index)` → Triggered via keyboard or double-click.
- `entered(index)` → Triggered when mouse hovers over a cell.
- `pressed(index)` → Emitted when mouse is pressed on a cell.

Example Usage of Signals

```
1 def on_cell_clicked(index):
2     print("Clicked cell at row:", index.row(), "column:",
3         index.column())
4 table.clicked.connect(on_cell_clicked) # clicked
5
```

Integration with Other Widgets

Often paired with:

- `QStandardItemModel` for editable rows and columns.
- `QPushButton` for row operations (add/remove/update).

- `QLineEdit`, `QComboBox`, or `QSpinBox` for inline cell editors.

```
1 button.clicked.connect(lambda: model.setItem(0, 1,
2   QStandardItem("Updated")))
```

Styling

`QTableView` supports custom styling for table cells, headers, grid, and selection.

```
1 table.setStyleSheet("""
2   QTableView {
3       background-color: #2e2e2e;
4       color: #f1f1f1;
5       gridline-color: #444;
6       font-size: 13px;
7   }
8   QHeaderView::section {
9       background-color: #3c3c3c;
10      padding: 4px;
11      font-weight: bold;
12      border: 1px solid #222;
13  }
14  QTableView::item:selected {
15      background-color: #007acc;
16  }
17  """)
18
```

Common Pitfalls

- Not using a proper model — data won't display.
- Forgetting `resizeColumnsToContents()` → causes hidden or clipped data.
- Not handling signals with `index.data()` → accessing invalid values.
- Using fixed column widths in dynamic UIs may cause layout issues.

Best Practices

- Always assign a model first before trying to interact with the view.
- Use `QStandardItemModel` for simple editable tables.
- Use `QAbstractItemModel` for advanced/connected data structures.
- Resize columns programmatically for better UX.
- Keep styles consistent and clean for readability.

QTreeView

Purpose

- `QTreeView` is used to display **hierarchical data structures** in a tree-like layout.
- It is a part of the **Model/View framework** and separates data from presentation.
- Ideal when working with **nested lists**, **file trees**, or **parent-child relationships**.

In VFX Tools, use it for:

- Displaying **asset hierarchies** (e.g., Character > Rig > Controls).
- Visualizing **shot breakdowns** with nested tasks or subcomponents.
- Managing **directory structures**, version trees, or render branches.

Official Qt Documentation:

 [QTreeView — Qt for Python](#)

Key Features

- Displays data in expandable/collapsible tree form.
- Supports models like `QStandardItemModel` or `QFileSystemModel`.
- Allows editing, drag & drop, selection, and sorting.
- Full control over columns, headers, and appearance.
- Automatically syncs with the underlying model.

Key Components

- **Model:** Supplies data (usually with parent-child relationships).
- **View:** QTreeView visualizes the model in tree form.
- **Items:** Nodes in the tree (may have children or not).
- **Header:** Column labels shown at the top.

Common Methods

- `setModel(model)` → Sets the data model.
- `model()` → Returns the current model.
- `setEditTriggers(triggers)` → Controls editability of items.
- `setSelectionMode(mode)` → Defines how items are selected.
- `setSelectionBehavior(behavior)` → Row or item selection.
- `expand(index)` → Expands a specific item.
- `collapse(index)` → Collapses a specific item.
- `expandAll()` / `collapseAll()` → Expands or collapses the full tree.
- `header()` → Accesses the header view.
- `setColumnWidth(column, width)` → Sets width of a column.
- `resizeColumnToContents(column)` → Auto-sizes a column.

Example Usage of All Methods

```
1 tree = QTreeView()
2 tree.setModel(model)                                # setModel
3 tree.setEditTriggers(QTreeView.DoubleClicked)        # setEditTriggers
4 tree.setSelectionMode(QTreeView.SingleSelection)    # setSelectionMode
5 tree.setSelectionBehavior(QTreeView.SelectRows)      #
   setSelectionBehavior
6 tree.expandAll()                                     # expandAll
7 tree.collapseAll()                                   # collapseAll
8 tree.setColumnWidth(0, 200)                          # setColumnWidth
9 tree.resizeColumnToContents(1)                        #
   resizeColumnToContents
```

Signals and Slots

- `clicked(index)` → Emitted when an item is clicked.
- `doubleClicked(index)` → Emitted on double-click.
- `expanded(index)` → Emitted when an item is expanded.
- `collapsed(index)` → Emitted when an item is collapsed.

Example Usage of Signals

```

1 def on_item_clicked(index):
2     print("Clicked item:", index.data())
3
4 def on_item_expanded(index):
5     print("Expanded:", index.data())
6
7 tree.clicked.connect(on_item_clicked)      # clicked
8 tree.expanded.connect(on_item_expanded)    # expanded
9

```

Integration with Other Widgets

Often integrated with:

- `QStandardItemModel`, `QFileSystemModel`
- `QPushButton` (e.g., to expand/collapse or add/remove items)
- `QLabel` for selected item info

```

1 button.clicked.connect(tree.expandAll) # Expand all on button click

```

Styling

`QTreeView` supports full styling for rows, selection, and headers.

```

1 tree.setStyleSheet("""
2 QTreeView {
3     background-color: #2b2b2b;
4     color: #ffffff;
5     font-size: 13px;
6     border: 1px solid #444;
7 }
8 QTreeView::item:selected {
9     background-color: #007acc;
10 }
11 QHeaderView::section {
12     background-color: #3c3c3c;
13     font-weight: bold;
14 }
15 """)
16

```

Common Pitfalls

- Not expanding items → child data appears hidden.
- Forgetting to use a tree-compatible model (needs parent/child structure).

- Misusing index → always use `index.data()` to access item text.
- Skipping model setup → tree view appears blank.

Best Practices

- Use `QStandardItemModel` for custom tree structures.
- Set clear headers and column sizes for readability.
- Call `expandAll()` initially only if UX demands it.
- Provide visual feedback for expansion with `expanded` signal.
- Keep styles readable and consistent with the overall UI.